

Module 5

Procedural Testbench Writing

Learning objectives

- Procedural Constructs: Master ``initial`` and ``always`` blocks in Verilog, and control flow in C++
- Timing Control: Understand delay control, event control, and wait statements
- Reusable Routines: Create tasks/functions in Verilog and functions/classes in C++ for code organiza...
- File I/O: Read test vectors from files and write results for automated testing
- Test Sequences: Build complex, multi-step test sequences
- Advanced Patterns: Implement state machines, protocols, and transaction-based testbenches

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Procedural Constructs"]

T2["Timing Control"]

T1 --> T2

T3["Test Sequences"]

T2 --> T3

Master procedural testbench construction using Verilog procedural blocks and C++ control flow

Overview

- This module covers procedural testbench writing using Verilog's procedural blocks and C++ control s...

Design architecture

1. Protocol-oriented DUT (Module 5)

- ``dut/uart/simple_uart.v``: TX FSM
 - idle, start, data bits, stop;
 - ``tx_done`` handshake
- Clock domain: Single ``clk`` with active-low ``rst_n``; serial ``tx`` output
- Future DUTs: SPI/I2C examples extend the same procedural TB patterns
- Interface contract: ``tx_start`` pulse, ``tx_data[7:0]``, wait for ``tx_done`` before next byte

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  INIT[initial blocks] --> TASKS[tasks / functions]
  ALWAYS[always blocks] --> CLKDOM[clock domains]
  TASKS --> DUT[ALU / file I/O DUT]
  FILE[$fopen vectors] --> TASKS
  DUT --> LOG[results file]
```

2. Procedural testbench control architecture

- Verilog: ``initial`` sequences
call tasks; ``always`` blocks for
clock and monitors
- C++: ``main`` or test class
methods encode multi-step
protocols with loops and waits
- Timing: ``#delay`` (Verilog) or
counted ``eval()`` cycles
(Verilator) align to bit times
- State alignment: TB FSM tracks
UART phases while DUT FSM runs
in RTL

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
SEQ[test sequences] --> ROUT[task_function routines]
ROUT --> DUT[drive + check]
DUT --> FILEIO[file-based regression]
FILEIO --> RPT[pass/fail log]

3. Reusable routine layer

- Tasks/functions: ``send_byte``, ``wait_tx_done``, ``apply_reset`` — shared across tests
- C++ helpers: Same operations as methods on a ``UartDriver`` helper class
- File-driven layer: Sequences can be loaded from vector files for long regressions

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
PROC[procedural_constructs] --> REUSE[reusable_routines]
REUSE --> FIO[file_io_regression]
FIO --> TSEQ[test_sequences]
TSEQ --> REG[module5.sh --check]
```

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

INIT[initial blocks] --> TASKS[tasks / functions]

ALWAYS[always blocks] --> CLKDOM[clock domains]

TASKS --> DUT[ALU / file I/O DUT]

FILE[\$fopen vectors] --> TASKS

DUT --> LOG[results file]

Module 5: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

SEQ[test sequences] --> ROUT[task_function routines]

ROUT --> DUT[drive + check]

DUT --> FILEIO[file-based regression]

FILEIO --> RPT[pass/fail log]

Module 5: stimulus, observation, and checking.

Task/function Makefile

```
task_function_test_verilog: task_function_test_verilog.v
    @echo "=== Compiling Verilog task/function example ==="
    $(IVERILOG) -o task_function_test task_function_test_verilog.v
$(DUT_DIR)/alus/simple_alu.v
    @echo "=== Running simulation ==="
    $(VVP) task_function_test
```

Task — apply stimulus and check

```
task test_alu_operation;
    input [7:0] a_val, b_val;
    input [1:0] op_val;
    begin
        // Get operation name from op code
        task_op_name = get_op_name(op_val);

        // Step 1: Apply test vector to DUT inputs
        // This drives the DUT with the test stimulus
        a = a_val;
        b = b_val;
        op = op_val;

        // Step 2: Wait for combinational logic to propagate
        // #5 delay ensures output has stabilized before checking
        // For sequential logic, would wait for clock edge instead
    end
endtask
```

...

Function — expected ALU result

```
function [7:0] calculate_expected;
    input [7:0] a_val, b_val;
    input [1:0] op_val;
    begin
        // Case statement implements operation selection
        // This is a common pattern for ALU operations
        case (op_val)
            2'b00: calculate_expected = a_val + b_val;    // ADD:
arithmetic addition
            2'b01: calculate_expected = a_val - b_val;    // SUB:
arithmetic subtraction
            2'b10: calculate_expected = a_val & b_val;    // AND:
bitwise AND
            2'b11: calculate_expected = a_val | b_val;    // OR:
bitwise OR
            default: calculate_expected = 8'h00;    // Default:
        return
        0 for invalid op
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- `sim_main.cpp`: generates `clk/rst_n`, calls `eval()` until `$finish`
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: `module5/examples/<example>/` — run `make run` from that directory

Directed test execution sequence (1)

- task/function computes expected
- compare DUT output
- write pass/fail to log file
- Define tasks/functions
- call from initial block

Directed test execution sequence (2)

- load file vectors
- log results
- `./scripts/module5.sh --check`

File I/O regression build

```
# Makefile for file I/O examples

IVERILOG = iverilog
VVP = vvp
VERILATOR = verilator
DUT_DIR = ../../../../module4/dut

all: file_based_test_verilog file_based_test_cpp

file_based_test_verilog: file_based_test_verilog.v test_vectors.hex
expected_results.hex
    @echo "=== Compiling Verilog file-based testbench ==="
    $(IVERILOG) -o file_based_test file_based_test_verilog.v
$(DUT_DIR)/alus/simple_alu.v
    @echo "=== Running simulation ==="
    $(VVP) file_based_test
    @echo "=== Checking output file ==="
```

File read and test loop

```
/**
 * Main Test Sequence
 *
 * This initial block orchestrates the file-based testbench:
 * 1. Read test vectors from input files
 * 2. Open output file for results
 * 3. Apply test vectors and check results
 * 4. Write results to file
 * 5. Close file and finish
 *
 * File I/O in Verilog:
 * - $readmemh: Read hex format file into memory array
 * - $readmemb: Read binary format file into memory array
 * - $fopen: Open file for writing (returns file handle)
 * - $fwrite: Write formatted data to file
 * - $fclose: Close file handle
```

```
# ...
```

Test sequence orchestration

```
begin

    $display("=====");
    $display("Running ALU Test Suite");
    $display("=====");

    // Test ADD
    test_alu_operation(8'h10, 8'h20, 2'b00);
    test_alu_operation(8'hFF, 8'h01, 2'b00);

    // Test SUB
    test_alu_operation(8'h30, 8'h10, 2'b01);
    test_alu_operation(8'h00, 8'h01, 2'b01);

    // Test AND
    test_alu_operation(8'hAA, 8'h55, 2'b10);
    test_alu_operation(8'hFF, 8'hFF, 2'b10);
```

...

Verification & testing methods

1. Sequence-based stimulus

- Multi-step tests: Reset →
configure → send N bytes → check
`tx` bit timing
- Handshaking: Poll `tx\_done` or
use timeout counters to detect
stuck FSM
- Negative tests: Start without
reset, back-to-back `tx\_start` —
expect defined errors

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
PROC[procedural_constructs] --> REUSE[reusable_routines]
REUSE --> FIO[file_io_regression]
FIO --> TSEQ[test_sequences]
TSEQ --> REG[module5.sh --check]
```

2. Timing and synchronization checks

- Bit period: Verify one `eval()` or time step per bit cell in examples
- Setup/hold: Ensure `tx\_data` stable before `tx\_start` per README contracts
- Waveform review: UART frame visible on `tx` in VCD — start bit, LSB-first data

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
PROC[procedural_constructs] --> REUSE[reusable_routines]
REUSE --> FIO[file_io_regression]
FIO --> TSEQ[test_sequences]
TSEQ --> REG[module5.sh --check]
```


3. Reuse and regression methodology

- Routine library: Centralize protocol steps; tests only describe high-level scenarios
- File I/O regression: Replay captured stimulus files across tool releases
- Orchestration:
`./scripts/module5.sh --test-sequences` and `--file-io` for focused runs

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
  PROC[procedural_constructs] --> REUSE[reusable_routines]
  REUSE --> FIO[file_io_regression]
  FIO --> TSEQ[test_sequences]
  TSEQ --> REG[module5.sh --check]
```

Procedural control — initial/always

```
/**
 * Task and Function Example - Verilog
 *
 * Demonstrates:
 * - Task definitions for test sequences
 * - Function definitions for calculations
 * - Parameter passing
 * - Reusable verification routines
 *
 * Usage:
 *   iverilog -o task_function_test task_function_test_verilog.v
 *   ../../dut/alus/simple_alu.v
 *   vvp task_function_test
 */

`timescale 1ns/1ps
```

Result logging to file

```
* $readmemh reads hexadecimal values from file into memory array.
    * File format: One hex value per line (e.g., "aabbcc" for
24-bit value)
    *
    * test_vector format: [a[7:0], b[7:0], op[1:0]] = 18 bits
total
    * - Bits [17:10]: Input A (8 bits)
    * - Bits [9:2]: Input B (8 bits)
    * - Bits [1:0]: Operation code (2 bits)
    *
    * expected format: Expected result value (8 bits)
    */
$readmemh("test_vectors.hex", test_vector);
$readmemh("expected_results.hex", expected);

$display("Read test vectors from file");
// Note: $size() is not supported by iverilog for memory arrays
// Using a fixed size or counting manually would be needed
```

Syllabus topics

1. Procedural Constructs (1/6)

- Verilog: Initial Blocks for Test Sequences
 - Initial block usage
 - Sequential execution
 - Multiple initial blocks
 - Initial block organization
- Verilog: Always Blocks for Continuous Behavior

1. Procedural Constructs (2/6)

- Always block usage
- Continuous execution
- Clock generation
- Continuous monitoring
- C++: Main Function and Control Flow
- Main function structure

1. Procedural Constructs (3/6)

- Control flow
- Sequential execution
- Function organization
- C++: Loops and Conditionals
- For loops
- While loops

1. Procedural Constructs (4/6)

- If-else statements
- Switch statements
- Verilog: Blocking vs. Non-Blocking Assignments in Testbenches
- Blocking assignments (=)
- Non-blocking assignments (<=)
- When to use each

1. Procedural Constructs (5/6)

- Best practices
- Procedural Timing Control (Both Paradigms)
- Timing control methods
- Timing accuracy
- Timing relationships
- Timing best practices

1. Procedural Constructs (6/6)

- Verilog: Event Control (@, wait)
- Event control (@)
- Wait statements
- Event-based execution
- Event coordination

2. Timing Control (1/7)

- Verilog: Delay Control (# delays)
- Delay syntax
- Delay timing
- Delay accuracy
- Delay best practices
- ****Verilog: Event Control (@posedge, @negedge, @*)****

2. Timing Control (2/7)

- Posedge control
- Negedge control
- Wildcard control
- Event-based timing
- C++: Time Management in Simulation Loop
- Simulation loop

2. Timing Control (3/7)

- Time tracking
- Time advancement
- Time management
- C++: Clock Cycle Counting
- Cycle counting
- Cycle-based timing

2. Timing Control (4/7)

- Cycle accuracy
- Cycle management
- Verilog: Wait Statements
- Wait syntax
- Wait conditions
- Wait timing

2. Timing Control (5/7)

- Wait best practices
- Timing Accuracy (Both Paradigms)
- Timing precision
- Timing relationships
- Timing verification
- Timing debugging

2. Timing Control (6/7)

- Race Condition Avoidance
- Race conditions
- Avoidance techniques
- Best practices
- Debugging
- ``timing_control_verilog.v``: Delay control, event control, wait statements

2. Timing Control (7/7)

- ``timing_control_cpp.cpp``: Time management in simulation loop

3. Test Sequences (1/5)

- Sequential Test Application
- Test ordering
- Sequential execution
- Test dependencies
- Test coordination
- Test Step Implementation

3. Test Sequences (2/5)

- Step definition
- Step execution
- Step timing
- Step verification
- Sequence Timing
- Sequence timing

3. Test Sequences (3/5)

- Timing control
- Timing accuracy
- Timing verification
- Synchronization with DUT
- DUT synchronization
- Timing relationships

3. Test Sequences (4/5)

- Synchronization methods
- Synchronization verification
- Sequence Verification
- Sequence checking
- Sequence validation
- Sequence debugging

3. Test Sequences (5/5)

- Sequence best practices

4. Reusable Routines (1/6)

- Verilog: Task Definitions for Test Sequences
 - Task syntax
 - Task usage
 - Task parameters
 - Task organization
- Verilog: Function Definitions for Calculations

4. Reusable Routines (2/6)

- Function syntax
- Function usage
- Function return values
- Function organization
- C++: Function Definitions for Test Sequences
- Function syntax

4. Reusable Routines (3/6)

- Function usage
- Function parameters
- Function organization
- C++: Class Methods for Organization
- Class methods
- Method organization

4. Reusable Routines (4/6)

- Method reusability
- Method best practices
- Parameter Passing (Both Paradigms)
- Parameter syntax
- Parameter types
- Parameter passing

4. Reusable Routines (5/6)

- Parameter best practices
- Reusable Verification Routines
- Routine design
- Routine organization
- Routine reusability
- Routine library

4. Reusable Routines (6/6)

- ``task_function_test_verilog.v``: Tasks and functions for reusable test sequences
- ``function_class_test_cpp.cpp``: Functions and classes for reusable test sequences

5. Loops and Control Flow (1/5)

- For Loops in Testbenches
- For loop syntax
- For loop usage
- Loop-based testing
- Loop best practices
- While Loops

5. Loops and Control Flow (2/5)

- While loop syntax
- While loop usage
- Conditional loops
- Loop termination
- Repeat Loops
- Repeat syntax

5. Loops and Control Flow (3/5)

- Repeat usage
- Fixed iteration
- Repeat best practices
- Conditional Execution
- If-else statements
- Switch statements

5. Loops and Control Flow (4/5)

- Conditional logic
- Conditional best practices
- Loop-Based Test Generation
- Test generation
- Loop-based patterns
- Generation strategies

5. Loops and Control Flow (5/5)

- Generation best practices

6. File I/O (1/7)

- Verilog: File Reading (\$readmemh, \$readmemb)
- \$readmemh syntax
- \$readmemb syntax
- File reading
- Memory initialization
- Verilog: File Writing (\$fopen, \$fwrite, \$fclose)

6. File I/O (2/7)

- \$fopen syntax
- \$fwrite syntax
- \$fclose syntax
- File writing
- C++: File I/O (fstream, ifstream, ofstream)
- fstream usage

6. File I/O (3/7)

- ifstream usage
- ofstream usage
- File operations
- C++: Reading Test Vectors
- Vector reading
- Vector parsing

6. File I/O (4/7)

- Vector storage
- Vector application
- C++: Writing Results
- Result writing
- Result formatting
- Result storage

6. File I/O (5/7)

- Result analysis
- Test Vector File Formats
- Format types
- Format selection
- Format parsing
- Format best practices

6. File I/O (6/7)

- Result Logging to Files
- Logging methods
- Logging formats
- Logging organization
- Logging best practices
- File-Based Testbenches (Both Paradigms)

6. File I/O (7/7)

- File-based design
- File-based patterns
- File-based best practices
- File-based examples
- ``file_based_test_verilog.v``: File-based testbench using `$readmemh` and `$fwrite`
- ``file_based_test_cpp.cpp``: File-based testbench using C++ file streams

7. Advanced Procedural Patterns (1/6)

- State Machine-Based Testbenches
- State machine design
- State transitions
- State verification
- State machine patterns
- Protocol-Based Testbenches

7. Advanced Procedural Patterns (2/6)

- Protocol modeling
- Protocol sequences
- Protocol verification
- Protocol patterns
- Transaction-Based Patterns
- Transaction modeling

7. Advanced Procedural Patterns (3/6)

- Transaction sequences
- Transaction verification
- Transaction patterns
- Layered Testbench Structure
- Layer organization
- Layer interaction

7. Advanced Procedural Patterns (4/6)

- Layer verification
- Layer patterns
- Location: (Coming soon)
- Demonstrates: Protocol testbench, procedural sequences
- Location: (Coming soon)
- Demonstrates: Protocol testbench, C++ control flow

7. Advanced Procedural Patterns (5/6)

- Location: ``module5/examples/file_io/``
- Demonstrates: File I/O, test vector reading, result writing
- Location: ``module5/examples/reusable_routines/task_function_test_verilog.v``
- Demonstrates: Tasks and functions for reusable test sequences
- Location: ``module5/examples/reusable_routines/function_class_test_cpp.cpp``
- Demonstrates: Functions and classes for reusable test sequences

7. Advanced Procedural Patterns (6/6)

- Location: (Coming soon)
- Demonstrates: State machine-based testbench patterns
- Location: (Coming soon)
- Demonstrates: Protocol testbench patterns

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --help
```

Terminal: module5_check

Usage: ./scripts/module5.sh [OPTIONS]

Module 5: Procedural Testbench Writing

This script runs examples and tests for Module 5.

OPTIONS:

Examples:

- procedural-constructs Run procedural constructs examples
- timing-control Run timing control examples
- test-sequences Run test sequence examples
- reusable-routines Run reusable routines examples
- file-io Run file I/O examples
- advanced-patterns Run advanced procedural patterns
- all-examples Run all examples (default)
- skip-examples Skip all examples

Tests:

- verilog-tests Run Verilog testbenches
- cpp-tests Run C++ testbenches
- all-tests Run all tests (default)
- skip-tests Skip all tests

Environment:

- jobs N Number of parallel build jobs (default: 8)
- no-clean Skip cleaning before build (faster rebuilds)

Other:

- help, -h Show this help message

EXAMPLES:

```
# Run all examples and tests
./scripts/module5.sh
```

```
# Run only file I/O examples
./scripts/module5.sh --file-io
```


Exercise scaffold

```
./scripts/module5.sh --scaffold
```

Demo: Procedural constructs

```
$ cd module5/examples/procedural_constructs && make all
```

Terminal: ex01_procedural_constructs

```
=====
Procedural Constructs Examples
=====
Procedural constructs examples are coming soon.
See module5/README.md for details.
=====
```

Demo: Reusable routines

```
$ cd module5/examples/reusable_routines && make all
```

Terminal: ex02_reusable_routines

```
=== Compiling Verilog task/function example ===
iverilog -o task_function_test task_function_test_verilog.v ../../module4/dut/alus/simple_alu
=== Running simulation ===
vvp task_function_test
=====
Running ALU Test Suite
=====
VCD info: dumpfile task_function_test.vcd opened for output.
[PASS] Test 1: ADD a=0x10, b=0x20, result=0x30
[PASS] Test 2: ADD a=0xff, b=0x01, result=0x00
[PASS] Test 3: SUB a=0x30, b=0x10, result=0x20
[PASS] Test 4: SUB a=0x00, b=0x01, result=0xff
[PASS] Test 5: AND a=0xaa, b=0x55, result=0x00
[PASS] Test 6: AND a=0xff, b=0xff, result=0xff
[PASS] Test 7: OR a=0xaa, b=0x55, result=0xff
[PASS] Test 8: OR a=0x00, b=0x00, result=0x00

=====
Test Summary
=====
Total tests: 8
Passed:      8
Failed:      0
=====
✓ All tests PASSED!
=====
task_function_test_verilog.v:226: $finish called at 50000 (1ps)
=== Compiling C++ function/class example ===
verilator --cc --exe --build -I../../module4/dut/alus \
[../../module4/dut/alus/simple_alu.v function_class_test_cpp.cpp
make: verilator: No such file or directory
make: *** [Makefile:18: function_class_test_cpp] Error 127
```

Demo: File I/O

```
$ cd module5/examples/file_io && make all
```

Terminal: ex03_file_io

```
=== Compiling Verilog file-based testbench ===
iverilog -o file_based_test file_based_test_verilog.v ../../module4/dut/alus/simple_alu.v
=== Running simulation ===
vvp file_based_test
=====
File-Based Testbench Example
=====
WARNING: file_based_test_verilog.v:85: $readmemh(test_vectors.hex): Too many words in the file f
Read test vectors from file
Test 1: a=0x00, b=0x04, op=00, result=0x04, expected=0x30 [FAIL]
ERROR: file_based_test_verilog.v:132: Test 1 failed
      Time: 5000 Scope: file_based_test_verilog
Test 2: a=0x00, b=0x08, op=00, result=0x08, expected=0x00 [FAIL]
ERROR: file_based_test_verilog.v:132: Test 2 failed
      Time: 10000 Scope: file_based_test_verilog
Test 3: a=0x00, b=0x00, op=00, result=0x00, expected=0x20 [FAIL]
ERROR: file_based_test_verilog.v:132: Test 3 failed
      Time: 15000 Scope: file_based_test_verilog
Test 4: a=0x00, b=0x3f, op=11, result=0x3f, expected=0xff [FAIL]
ERROR: file_based_test_verilog.v:132: Test 4 failed
      Time: 20000 Scope: file_based_test_verilog
Test 5: a=0x00, b=0x00, op=01, result=0x00, expected=0x00 [PASS]
Test 6: a=0x00, b=0x00, op=00, result=0x00, expected=0xff [FAIL]
ERROR: file_based_test_verilog.v:132: Test 6 failed
      Time: 30000 Scope: file_based_test_verilog
Test 7: a=0x00, b=0x0c, op=00, result=0x0c, expected=0xff [FAIL]
ERROR: file_based_test_verilog.v:132: Test 7 failed
      Time: 35000 Scope: file_based_test_verilog
Test 8: a=0x00, b=0x04, op=00, result=0x04, expected=0x00 [FAIL]
ERROR: file_based_test_verilog.v:132: Test 8 failed
      Time: 40000 Scope: file_based_test_verilog
Test 9: a=0x00, b=0x00, op=01, result=0x00, expected=0x66 [FAIL]
ERROR: file_based_test_verilog.v:132: Test 9 failed
      Time: 45000 Scope: file_based_test_verilog
Test 10: a=0x00, b=0x00, op=00, result=0x00, expected=0x00 [PASS]
```

Demo: Test sequences

```
$ cd module5/examples/test_sequences && make all
```

Terminal: ex04_test_sequences

```
=====
Test Sequence Examples
=====
Test sequence examples are coming soon.
See module5/README.md for details.
=====
```

Practice & assessment

What you should know (1/11)

- Write procedural testbenches effectively (both paradigms)
- Control timing in testbenches (Verilog and C++)
- Create complex test sequences (both paradigms)
- Use tasks/functions (Verilog) and functions/classes (C++) for organization
- Implement file-based testing (both paradigms)
- Build protocol testbenches (both paradigms)

What you should know (2/11)

- Define tasks for test sequences
- Use functions for calculations
- Organize code with reusable routines
- Define functions for test sequences
- Use classes for organization
- Compare with Verilog approach

What you should know (3/11)

- Read test vectors from files
- Write results to files
- Implement file-based testing
- Implement protocol sequences
- Handle timing requirements
- Verify protocol compliance

What you should know (4/11)

- Model testbench as state machine
- Implement state transitions
- Verify state coverage
- Design reusable components
- Parameterize routines
- Build verification library

What you should know (5/11)

- Can write procedural testbenches effectively (both paradigms)
- Can control timing in testbenches (Verilog and C++)
- Can create complex test sequences (both paradigms)
- Can use tasks/functions (Verilog) and functions/classes (C++) for organization
- Can implement file-based testing (both paradigms)
- Can build protocol testbenches (both paradigms)

What you should know (6/11)

- Module 6: SystemVerilog Testbench Features - Learn SystemVerilog enhancements
- Module 7: Coverage and Assertions - Master coverage and assertion-based verification
- File Headers: Every file includes comprehensive header documentation:
 - Purpose and learning objectives
 - Key concepts explained
 - Compilation and execution instructions

What you should know (7/11)

- Usage examples
- Inline Comments: Detailed comments explain:
- Why code is written a certain way (not just what it does)
- Procedural block behavior and timing
- Task/function usage and parameter passing
- File I/O operations and formats

What you should know (8/11)

- Timing control mechanisms
- Code Organization: Comments group related code:
- Signal declarations
- DUT instantiation
- Reusable routines (tasks/functions)
- Test sequences

What you should know (9/11)

- File operations
- Icarus Verilog Documentation:
<http://iverilog.wikia.com/>
- Verilator Documentation: <https://verilator.org/>
- GTKWave Documentation:
<http://gtkwave.sourceforge.net/>
- IEEE 1364-2005 Standard: Verilog Hardware Description Language
- IEEE 1800-2017 Standard: SystemVerilog Language Reference Manual

What you should know (10/11)

- Universal Verification Methodology (UVM):
<https://accellera.org/downloads/standards/uvm>
- UVM Core Repository: <https://github.com/universal-verification-methodology/core>
- Official UVM library with examples of:
 - Reusable components (agents, drivers, monitors)
 - Sequence-based stimulus generation
 - Configuration management

What you should know (11/11)

- Phase-based testbench control

Assessment checklist

- Can write procedural testbenches effectively (both paradigms)
- Can control timing in testbenches (Verilog and C++)
- Can create complex test sequences (both paradigms)
- Can use tasks/functions (Verilog) and functions/classes (C++) for organization
- Can implement file-based testing (both paradigms)
- Can build protocol testbenches (both paradigms)

Summary & next steps

- Master procedural testbench construction using Verilog procedural blocks and C++ control flow
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: Next module in course